2520030117
skill-06

EXPERIMENT – 6

Title:

Process Escape Sequences and Child Process Execution

Aim:

To implement process escape sequences, handle escaped spaces and special symbols, create child processes, execute programs, and test command launching.

Objective:

- To handle escaped spaces.
- To handle escaped special symbols.
- To preserve escaped characters.
- To validate parser output.
- To create child processes.
- To execute programs.
- To pass arguments to programs.
- To handle execution errors.
- To manage the parent process.

Theory:

An escape sequence allows special characters to be treated as normal characters.

For example:

echo hello\ world

The escaped space is treated as part of the same argument.

A shell can also create a **child process** using `fork()`. The child process can execute a program using `execvp()`, while the parent process can wait for the child using `wait()`.

Algorithm:

1. Start the shell.
2. Display the shell prompt.
3. Read the command from the user.
4. Check for escape characters.
5. Preserve escaped spaces and special symbols.
6. Parse the command into arguments.

7. Create a child process using `fork()`. 8. Execute the command using `execvp()`. 9. Display an error if execution fails.
10. Parent process waits for the child process. 11. Repeat for the next command.
12. Stop when the user enters `exit`.

Functions Used:

Function	Purpose
`fgets()`	Read command
`fork()`	Create child process
`execvp( )`	Execute program
`wait()`	Wait for child process
`printf( )`	Display output
`perror( )`	Display execution errors
`exit()`	Terminate process

Sample Input:

2520030340_kruthi> echo hello\ world

Sample Output:

hello world

Observation:

1. Escaped spaces were handled correctly. 2. Escaped special characters were preserved. 3. Child processes were created successfully.
4. Commands were executed using the child process.
5. Arguments were passed correctly.
6. Execution errors were handled.
7. The parent process waited for the child process.

Result:

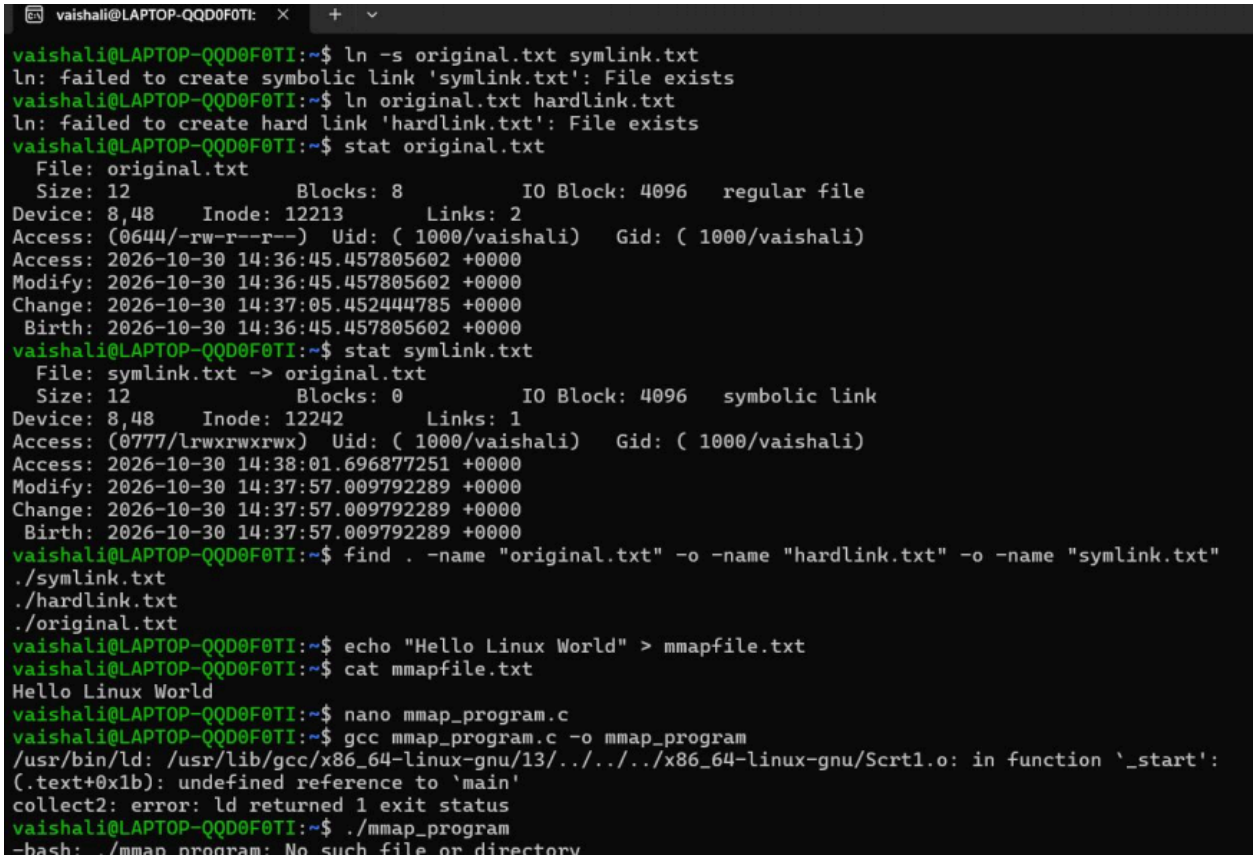
The shell successfully handled escape sequences and created child processes to execute

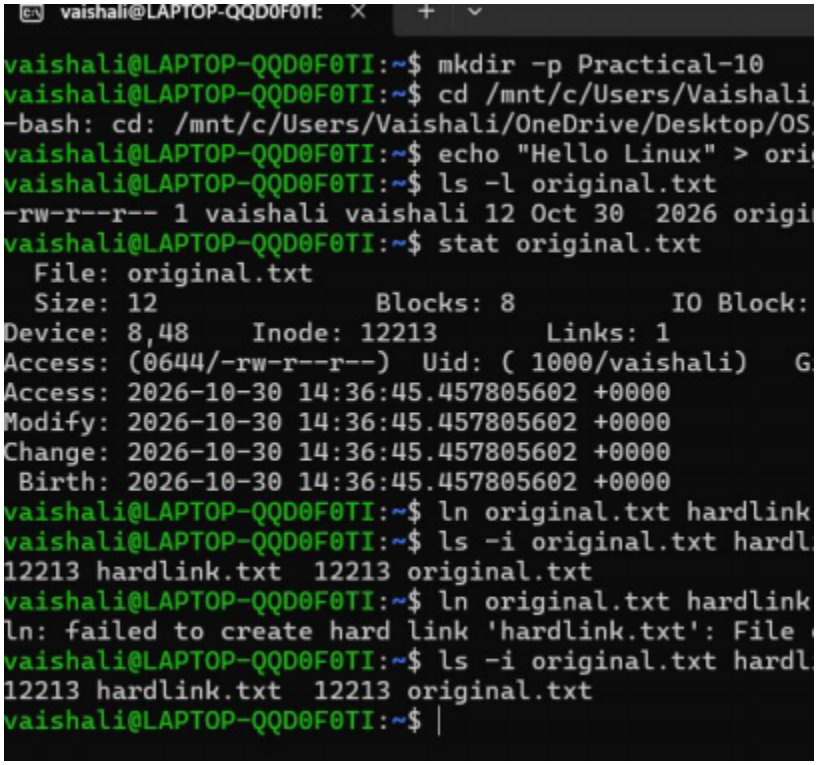
commands.

Conclusion:

The experiment successfully demonstrated **escape sequence handling and process creation** using `fork()`, `execvp()`, and `wait()`.

Output: Escape sequences, escaped spaces, special symbols, command execution, argument passing, and execution error handling.




vaishali@LAPTOP-QQD0F0TI:~$ mkdir -p Practical-10
vaishali@LAPTOP-QQD0F0TI:~$ cd /mnt/c/Users/Vaishali
-bash: cd: /mnt/c/Users/Vaishali/OneDrive/Desktop/OS
vaishali@LAPTOP-QQD0F0TI:~$ echo "Hello Linux" > ori
vaishali@LAPTOP-QQD0F0TI:~$ ls -l original.txt
-rw-r--r-- 1 vaishali vaishali 12 Oct 30 2026 origi
vaishali@LAPTOP-QQD0F0TI:~$ stat original.txt
File: original.txt
Size: 12 Blocks: 8 IO Block:
Device: 8,48 Inode: 12213 Links: 1
Access: (0644/-rw-r--r--) Uid: (1000/vaishali) G
Access: 2026-10-30 14:36:45.457805602 +0000
Modify: 2026-10-30 14:36:45.457805602 +0000
Change: 2026-10-30 14:36:45.457805602 +0000
Birth: 2026-10-30 14:36:45.457805602 +0000
vaishali@LAPTOP-QQD0F0TI:~$ ln original.txt hardlink
vaishali@LAPTOP-QQD0F0TI:~$ ls -i original.txt hardl
12213 hardlink.txt 12213 original.txt
vaishali@LAPTOP-QQD0F0TI:~$ ln original.txt hardlink
ln: failed to create hard link 'hardlink.txt': File
vaishali@LAPTOP-QQD0F0TI:~$ ls -i original.txt hardl
12213 hardlink.txt 12213 original.txt
vaishali@LAPTOP-QQD0F0TI:~$